

Analysis: Crop Triangles

The small case was easy to solve using brute force. For each combination of three points you needed to test if the center had integer coordinates. One tricky part was generating the sequence of points, since using 32-bit integers would have resulted in overflow when doing multiplications. One way to get around that problem was to use 64-bit integers.

The observation that helps in solving the large case is that we don't care about the range of the coordinates for the points in the input. We only care about the coordinates modulo 3. We have 9 different classes of points. In `bucket[i]` we will count the number of points for which $x \% 3 = i / 3$ and $y \% 3 = i \% 3$. There are three different ways of choosing 3 points out of the 9 classes of points: three points from the same class, two points from the same class and one from a different class and three points from three different classes.

If the three points are in the same class then it's obvious that the center will have integer coordinates. It is easy to see that there is no triangle with two points in the same class and one point in a different class.

Here is some code that implements this idea:

```
for (int i = 0; i < n; i++) {
    bucket[((int)X0 % 3) * 3 + (int)Y0 % 3]++;
    X0 = (A * X0 + B) % M;
    Y0 = (C * Y0 + D) % M;
}

// The first case.
for (int i = 0; i < 9; i++)
    // We use the formula for n choose 3 so that,
    // we don't use the same point twice or count
    // the same triangle more than once.
    ret += bucket[i] * (bucket[i]-1) * (bucket[i]-2) / 6;

// The third case.
for (int i = 0; i < 9; i++)
    for (int j = i + 1; j < 9; j++)
        for (int k = j + 1; k < 9; k++)
            if (((i / 3) + (j / 3) + (k / 3)) % 3 == 0) &&
                ((i % 3) + (j % 3) + (k % 3)) % 3 == 0)
                ret += bucket[i] * bucket[j] * bucket[k];
cout << "Case #" << prob++ << ": " << ret << endl;
```

Analysis: Mousetrap

In this nice problem, there are two viewpoints one can start with.

- The current position is fixed and the array of cards is rotating each time.
- Think of the cards as a fixed ring (represented by an array) and the current position in focus as a pointer moving on the ring.

For most of us (and our programs), it is more convenient to adopt the second viewpoint.

After reading the story, it is not hard to see that the task is clear: put card 1 in the first position, then for each card i (in the order 2, 3, ..., K), we start from the current position, and find the i -th empty spot to the right, wrap around as many times as necessary, then put card i there.

The problem is to simulate the process described above. What makes it interesting is that K can be as big as 1000000, which makes the naive $\Theta(K^2)$ simulation too slow for our 8-minute time limit. For each step, we need to compute the next position much faster than $\Theta(K)$ as in the naive approach. We describe three solutions below.

Solution A.

Let $S = \sqrt{K}$, we partition the K positions into S intervals of roughly equal size (also S). In addition to bookkeeping which positions are occupied (an array of size K , we call the first level counter), we also count for each interval how many positions are occupied (an array of size S that we call the second level counter). With this information, we may skip intervals of length S as many as possible, until we arrive at an interval where we know the card must belong to. Then in that interval we only need to deal with at most S first level counters.

Once we put down a card, it is a simple matter to update the counters. We only need to update one on the first level and one on the second level.

This solution runs in time $O(K^{1.5})$.

Solution B.

Push the idea in the previous solution further. Why not have more levels of counters? In fact, one nice plan is to organize the levels into a binary tree. On the bottom (first) level of the tree we have each position as a separate interval. Every time we go up one level, we combine every other interval with the next one. Thus we will have $\log K$ levels; the top level being a single interval with all the positions. We omit the details, since we will see this again in the analysis of a Round 1C problem. We mention that the total number of counters is $O(K)$, and for each card we will need $O(\log K)$ time to find the position and another $O(\log K)$ time to update the counters. The running time of this method is $O(K \log K)$.

For similar ideas in computer science, we refer to the Wiki page for [interval trees](#).

Solution C.

Now let us do something different. At each step, after one position is occupied by card number i , we delete the position from the deck.

Notice that n , the number of queries is at most 100. We do not need to relabel all the positions, it is enough to do this for those n that we are interested in.

The solution can be implemented in two flavors, based on which viewpoint in the beginning of the analysis you pick. The short C++ program is, again, based on the second one, where the position (`pos`) changes as a pointer, and the deck does not move, except we delete one position in each step.

```
for (int j = 0; j < n; j++) answers[j] = -1;
for (int i = 1, pos = 0; i <= K; i++) {
    // Compute the next position, after wrap-around.
    pos = (pos + i - 1) % (K - i + 1);
    for (int j = 0; j < n; j++)
        if (answers[j] < 0) {
            if (queries[j] == pos+1) {
                queries[j] = -1; answers[j] = i;
            } else if (queries[j] > pos+1) {
                // The effect of deleting the next position.
                queries[j]--;
            }
        }
}
```

You can use a trick to combine the two arrays `queries[]` and `answers[]` into one. The program runs in $\Theta(n K)$ time.

More information:

[Interval trees](#)

Analysis: Number Sets

This problem describes a process where integers are grouped into sets, and then we are asked how many sets there are in the final result.

The process described is fairly slow:

- Create singleton sets of integers.
- Take each pair of integers;
- factor the two integers and see if they have a prime factor in common that is greater than or equal to P ;
- if so, merge the sets that have these two integers.

We need to find the result of this process using a faster method. A few observations help here.

Firstly, we only need to find prime factors less than the size of the interval. If a prime is larger than or equal to the size of the interval, then at most one integer in the interval can have that prime as a factor, so it will never be used to merge sets.

Secondly, we can take a faster approach than finding the prime factors of each integer separately. Instead we consider each prime in turn and find all the integers in the interval which have this prime as a factor, a technique generally called a *sieve*.

Thirdly, joining sets can be implemented efficiently with a data structure called *union-find* (or the *disjoint sets data structure*). As we consider each prime and find all of the integers with that prime as a factor, we join all of the sets containing those integers. We could also build an undirected graph with nodes for the integers in the interval and the primes, and edges representing which integers are divisible by which primes; then the final sets are the connected components of this graph.

More information:

[Prime sieving](#) - [Union-Find](#) - [Connected components](#)